# L1: Lab Report-Pthread

## 1 Experiment Setup

Goal: describe the hardware and software configurations of the lab server, and fill out the following table

Hint: use Linux command

| Configuration | Configuration instructions | |
|---|---|---|
| Hardware configuration | CPU | Intel(R) Xeon(R) Gold 6330 CPU @ 2.00GHz |
| | Memory | 256 GiB |
| | Storage | 1.8 TB local disk (47 GB available), 57 TB network storage (Lustre filesystem) |
| Software configuration | Operating System | Ubuntu 20.04 (Linux kernel 5.4.0-208-generic) |
| | Compiler | g++ 9.4.0 |
| | Pthread version | POSIX Threads (NPTL, via glibc 2.31) |

## 2. Performance Evaluation of ShellSort

### 2.1 Performance of Sequential ShellSort

Goal: measure the performance of sequential ShellSort with different input array size

Hint: 1) use the code template to generate input arrays with 5/10/30 million integer elements, 2) measure the execution time of the sequential code template for each input array, and 3) report the execution time for each run

**Result:**

| Input Size (million) | Execution time (ns) |
|---|---|
| 5 | 925227924 |
| 10 | 2121189068 |
| 30 | 6979261956 |

### 2.2 Performance of Parallel ShellSort

Goal: measure the performance speed up of parallel ShellSort with different input array size

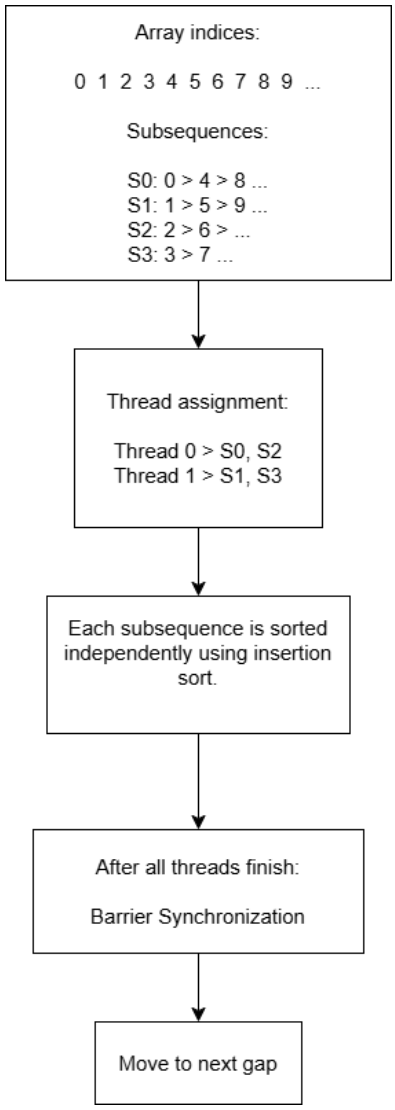
Hint: 1) use the code template to generate input arrays with 5/10/30 million integer elements, 2) measure the execution time of the parallel code template for each input array, 3) increase the thread number from [1,2,4,8,16,32,64], and report the execution time for each run, 4) calculate the performance speedup compared to sequential code (speedup = sequential execution time / parallel execution time), 5) use line chart to report the scalability for each input array when increasing the thread number from [1,2,4,8,16,32,64]

**Problem Partitioning:**

Shell Sort is parallelized by exploiting the independence of subsequences during the gap phases. For a given gap the array is divided into gap interleaved subsequences. Since each subsequence is sorted using a standard Insertion Sort and doesn't interact with others until the gap changes, we assign these subsequences to different threads.

## Diagram:

Example: gap = 4, threads = 2




## Parallel Features Used:

- POSIX Threads (pthread)
    - Thread creation: pthread_create
    - Synchronization: pthread_join
- Barrier Synchronization
    - pthread_barrier_wait

    - Ensures all threads complete current gap before proceeding
- Work Distribution
    - Static partitioning using thread ID (tid)

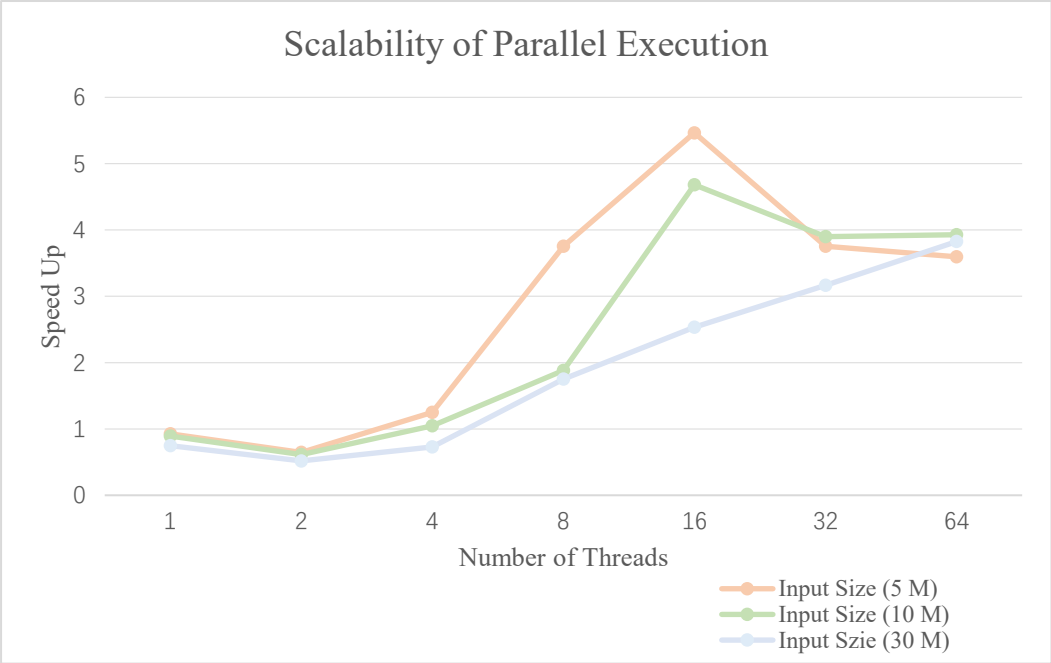
**Optimizations:**

1. Independent Subsequence Processing
    - No locks required (no shared writes within subsequence)
    - Minimizes synchronization overhead
2. Barrier Only at Gap Level
    - Synchronization happens once per gap, not per operation
3. Cache Locality
    - Threads repeatedly access elements with stride gap
    - Improves locality for small gaps
4. Load Distribution
    - start += m_nthreads balances subsequences across threads

**Result:**

| Input Size (million) | Threads | Execution Time (ns) | Speed Up |
|---|---|---|---|
| 5 | 1 | 997104776 | 0.9279 |
| | 2 | 1432540520 | 0.6459 |
| | 4 | 741710732 | 1.2474 |
| | 8 | 246501512 | 3.7534 |
| | 16 | 169302620 | 5.4649 |
| | 32 | 246325128 | 3.7561 |
| | 64 | 257314948 | 3.5957 |
| 10 | 1 | 2380364612 | 0.8911 |
| | 2 | 3460017964 | 0.6131 |
| | 4 | 2024713240 | 1.0476 |
| | 8 | 1126544820 | 1.8829 |
| | 16 | 453160992 | 4.6809 |
| | 32 | 544330076 | 3.8969 |

| | | | |
|---|---|---|---|
| | 64 | 539725812 | 3.9301 |
| 30 | 1 | 9357800864 | 0.7458 |
| | 2 | 13462428536 | 0.5184 |
| | 4 | 9598536704 | 0.7271 |
| | 8 | 3986928296 | 1.7505 |
| | 16 | 2753248376 | 2.5349 |
| | 32 | 2204032080 | 3.1666 |
| | 64 | 1824529768 | 3.8252 |




The speedup of the parallel Shell sort, measured against its sequential baseline, shows strong improvement as the number of threads increases, but only up to an optimal point. For smaller inputs (5M and 10M), the best speedup occurs at 16 threads, reaching about 5.46× (5M) and 4.68× (10M). For the larger input (30M), the speedup continues improving up to 64 threads, achieving around 3.83×. However, beyond the optimal thread count (e.g., 16 → 32 threads for smaller inputs), performance begins to decline or plateau.

In terms of efficiency (speedup ÷ threads), the results indicate moderate utilization at lower thread counts but rapid decline as threads increase. At 16 threads, efficiency is about 29–34%, showing relatively effective parallel execution. However, at 32 threads,

efficiency drops to roughly 10–12%, and further down to around 6% at 64 threads. This demonstrates diminishing returns as more threads are added.

The reason for this trend lies in the nature of Shell sort. When the gap is large, there are many independent subsequences, allowing threads to work in parallel efficiently, which explains the strong initial speedup. As the gap decreases, the number of subsequences reduces, limiting available parallelism and causing threads to become underutilized. Additionally, overhead from thread management, synchronization at each gap, and memory contention increases with more threads. These factors outweigh the benefits of parallelism at higher thread counts, leading to reduced efficiency and eventual performance degradation.

## 3. Performance Evaluation of RadixSort

### 3.1 Performance of Sequential RadixSort

#### 3.1.1 Implementation Details

Goal: describe your code implementation

Hint: use diagram or pseudo-code

The sequential radix sort is implemented using the Least Significant Digit (LSD) approach. Each 64-bit integer is processed 8 bits at a time, resulting in 256 buckets (0–255) per pass.

The algorithm performs 8 passes (since 64 / 8 = 8), sorting the array based on each byte from least significant to most significant.

For each pass:

1. Frequency Count: Scans the array to see how many times each byte value appears.
2. Prefix Sum (Accumulation): Transforms the count array into an index map, determining the starting position of each byte value in the temp array.
3. Stable Placement: Iterates backwards through the original array to place elements into temp. This preserves the relative order of elements with the same byte value, which is critical for Radix Sort to work.
4. Copy back to the original array

**Pseudocode:**

```
FUNCTION RadixSort(array, n):
    CREATE temp_array of size n

    FOR shift FROM 0 TO 56 STEP 8:
        RESET count_array[256] TO zeros

        // Step 1: Frequency counting
        FOR EACH element IN array:
            byte = (element >> shift) AND 0xFF
            INCREMENT count_array[byte]

        // Step 2: Determine positions (Prefix Sum)
        FOR i FROM 1 TO 255:
            count_array[i] = count_array[i] + count_array[i-1]

        // Step 3: Build output (Stable sort)
        FOR i FROM n-1 DOWN TO 0:
            byte = (array[i] >> shift) AND 0xFF
            POSITION = count_array[byte] - 1
            temp_array[POSITION] = array[i]
```

```
            DECREMENT count_array[byte]

        // Step 4: Update main array for next byte pass
        COPY temp_array TO array

    FREE temp_array
```

### 3.1.2 Performance Results

Goal: measure the performance of sequential RadixSort with different input array size

Hint: 1) use the code template to generate input arrays with 5/10/30 million integer elements, 2) measure the execution time of the sequential code template for each input array, and 3) report the execution time for each run

Result:

| Input Size (million) | Execution time (ns) |
|---|---|
| 5 | 230788340 |
| 10 | 488472640 |
| 30 | 1480803400 |

## 3.2 Performance of Parallel RadixSort

### 3.2.1 Implementation Details

Goal: describe your code implementation

Hint: use diagram or pseudo-code

The parallel implementation is based on a Least Significant Digit (LSD) Radix Sort, processing one bit at a time (base 2). Since the data type is uint64_t, the algorithm performs 64 passes, one for each bit position.

A data-parallel approach is used, where the input array is divided into T contiguous chunks (T = number of threads). Each thread independently processes its assigned portion, ensuring balanced workload distribution and good cache locality.

**Problem Partitioning:**

Radix sort is parallelized using data partitioning:

- Array divided into contiguous chunks
- Each thread processes its own chunk

```
Thread 0 -> [0 .... chunk-1]
Thread 1 -> [chunk ... 2* chunk-1]
...
Thread T -> [... remaining elements]
```

Each thread operates only on its own segment but collaborates with others during synchronization phases.

**Pipeline per Bit:**

For each bit position (0 to 63), the algorithm performs:

1. Local counting

   Each thread counts the number of 0s and 1s in its local chunk:

   - Results stored in shared arrays nzeros[tid] and nones[tid]

2. Global prefix sum computation

   Each thread computes its write positions:

   - Zero offset = sum of zeros from all lower thread IDs
   - One offset = total zeros + sum of ones from lower thread IDs

This determines the exact position in the output array.

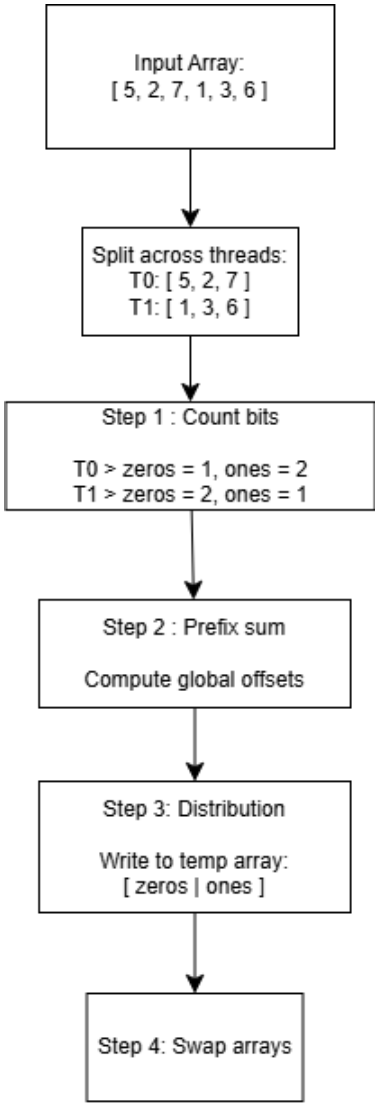
3. Parallel data redistribution

   Each thread writes elements into temp_array:

   - Elements with bit = 0 → placed in zero region
   - Elements with bit = 1 → placed in one region

4. Array Swapping

   - Thread 0 swaps m_array and temp_array
   - Prepares for the next bit iteration



**Pseudo-code:**

```
FUNCTION ParallelSort(array, n):
    INITIALIZE temp_array, nzeros[m_nthreads], nones[m_nthreads]
    INITIALIZE pthread_barrier

    SPAWN m_nthreads executing ThreadBody
    JOIN all threads

FUNCTION ThreadBody(thread_id):
    DETERMINE start and end indices based on thread_id

    FOR bit FROM 0 TO 63:
        // PHASE 1: LOCAL COUNT
        count 0s and 1s in array[start...end]
        STORE in nzeros[thread_id] and nones[thread_id]
        BARRIER_WAIT

        // PHASE 2: PREFIX SUM (OFFSET CALCULATION)
        offset0 = sum(nzeros[0...thread_id-1])
        offset1 = sum(nzeros[0...m_nthreads-1]) + sum(nones[0...thread_id-1])
        BARRIER_WAIT

        // PHASE 3: DISTRIBUTE
        FOR i FROM start TO end:
            IF bit of array[i] is 0:
                temp_array[offset0++] = array[i]
            ELSE:
                temp_array[offset1++] = array[i]
        BARRIER_WAIT
```

```
// PHASE 4: SWAP POINTERS
IF thread_id == 0:
    SWAP(m_array, temp_array)
BARRIER_WAIT
```

**Parallel Features Used:**

- POSIX Threads (pthread)
  - Parallel execution using multiple threads
- Barrier Synchronization
  - pthread_barrier_wait used after each phase
  - Ensures correctness and prevents race conditions
- Thread-local Computation
  - Local counting (nzeros, nones) reduces contention

**Optimization:**

1. Chunk-based Partitioning
   - Ensures contiguous memory access (cache-friendly)
2. Thread-local Counting
   - Avoids contention during counting phase
3. Prefix Sum without Locks
   - Uses deterministic accumulation across threads
4. Double Buffering
   - Swapping pointers (m_array ↔ temp_array)
   - Avoids costly data copying between passes

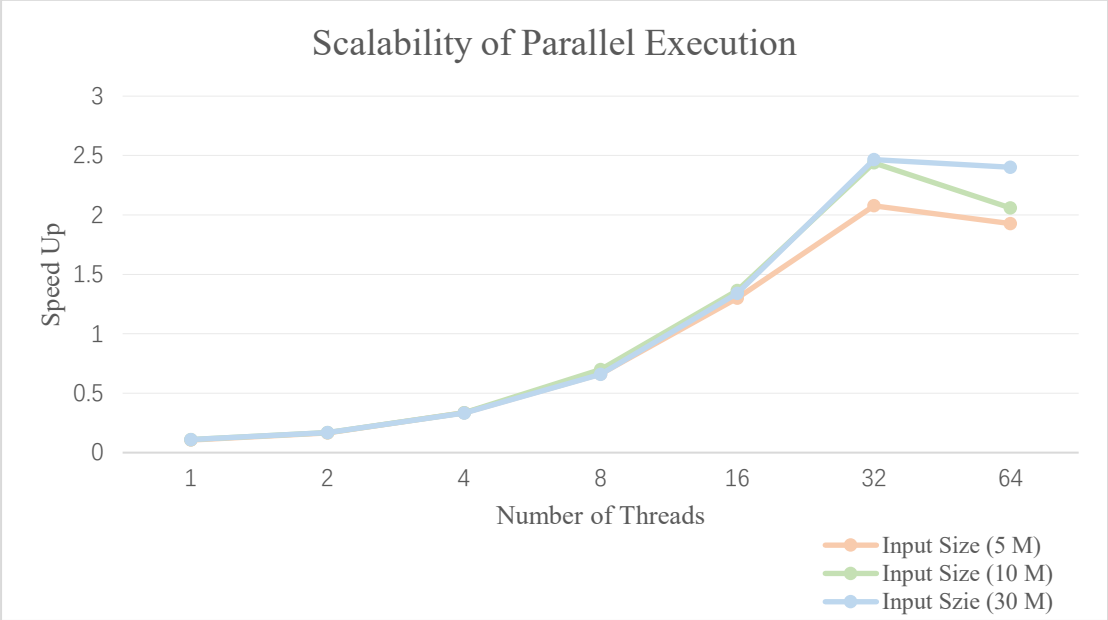
### 3.2.2 Performance Results

Goal: measure the performance speed up of parallel RadixSort with different input array size

Hint: 1) use the code template to generate input arrays with 5/10/30/ million integer elements, 2) measure the execution time of the parallel code template for each input array, 3) increase the thread number from [1,2,4,8,16,32,64], and report the execution time for each run, 4) calculate the performance speedup compared to sequential code (speedup = sequential execution time / parallel execution time), 5) use line chart to report the scalability for each input array when increasing the thread number from [1,2,4,8,16,32,64]

**Result:**

| Input Size (million) | Threads | Execution Time (ns) | Speed Up |
|---|---|---|---|
| 5 | 1 | 2181806860 | 0.1058 |
| | 2 | 1393268800 | 0.1656 |
| | 4 | 689569036 | 0.3347 |
| | 8 | 348332460 | 0.6626 |
| | 16 | 177623844 | 1.2993 |
| | 32 | 111117348 | 2.0770 |
| | 64 | 119667608 | 1.9289 |
| 10 | 1 | 4424575032 | 0.1104 |
| | 2 | 2880348296 | 0.1696 |
| | 4 | 1451622972 | 0.3365 |
| | 8 | 697470164 | 0.7003 |
| | 16 | 357897416 | 1.3648 |
| | 32 | 200360152 | 2.4380 |
| | 64 | 237241156 | 2.0590 |
| 30 | 1 | 13473844916 | 0.1099 |
| | 2 | 8811493528 | 0.1681 |
| | 4 | 4434297188 | 0.3339 |
| | 8 | 2248178480 | 0.6587 |
| | 16 | 1104996488 | 1.3401 |
| | 32 | 600573624 | 2.4656 |
| | 64 | 616260492 | 2.4029 |




The performance results show that the parallel radix sort initially performs significantly worse than the sequential version, especially at low thread counts. For example, at 1 thread, the execution time is much higher than the sequential implementation, resulting in a speedup of only around 0.1×. This indicates that the parallel implementation introduces substantial overhead, even without true parallelism.

As the number of threads increases, performance improves steadily. The speedup reaches approximately 2.0–2.5× at 32 threads across all input sizes (5M, 10M, and 30M). However, beyond this point (32 → 64 threads), the performance plateaus or slightly degrades, indicating limited scalability.

In terms of efficiency (speedup divided by the number of threads), the values are relatively low. At 32 threads, efficiency is around 6–8%, and it further decreases to approximately 3–4% at 64 threads. This demonstrates poor utilization of parallel resources and clear diminishing returns as more threads are added.

The main reason for this behavior lies in the design of the parallel radix sort. Unlike the sequential version, which processes 8 bits per pass (only 8 passes), the parallel implementation processes one bit per pass, resulting in 64 passes. Each pass involves

multiple synchronization points, including counting, prefix sum computation, data redistribution, and array swapping. This leads to a very high synchronization overhead.

Furthermore, the algorithm is memory-bound, as each pass requires reading from and writing to the entire array. As the number of threads increases, memory bandwidth becomes a bottleneck, limiting performance gains. The prefix sum step also introduces partial sequential dependency, which restricts scalability.

At low thread counts, the overhead of thread management and synchronization dominates execution time, making the parallel version slower than the sequential one. At higher thread counts, although parallelism helps reduce computation time, the benefits are offset by increased synchronization cost and memory contention. These factors explain why the speedup improves initially but quickly reaches a limit, resulting in low overall efficiency.